

PSD to UGUI Editor User Guide

This document explains how to use the PSD-to-UGUI editor and export pipeline. It focuses on the real runtime order implemented by the tool:

1. local image dedup inside the current export
2. representative node selection for each dedup group
3. slicing or nine-slice processing
4. common-directory dedup against shared PNG libraries
5. prefab generation and incremental update

What This Tool Produces

- PNG textures under the selected image export root, inside a subfolder named after the PSD
- Sprite import settings, including nine-slice borders when slicing succeeds
- A main UGUI prefab named after the PSD
- Optional standalone prefabs for selected nodes
- Config files that persist export settings, prefab mapping, dedup settings, and nine-slice settings

Important generated or persisted files:

- `Assets/PsdToUnityUI/PSDConfig/<PSDName>_export_config.json`
- `Assets/PsdToUnityUI/PrefabMap/<PSDName>_PrefabMap.json`
- `Assets/PsdToUnityUI/EditorConfig/PSD_CommonDirectories.json`
- `Assets/PsdToUnityUI/EditorConfig/PSD_DedupConfig.json`
- `Assets/PsdToUnityUI/EditorConfig/PSD_NineSliceConfig.json`

Recommended Workflow

Use this order when working on a PSD.

1. Configure global dedup and nine-slice settings from `Tools/PSD`.
2. Add shared texture folders to the common-directory list if you want cross-project reuse.
3. Open the PSD in `Tools/PSD/PSD Editor`.
4. Decide which nodes export as images, which nodes merge, and which nodes should become external prefabs or custom sprites.
5. For visually identical images, leave `Same-export dedup` enabled and mark one or more `Primary dedup node` candidates if you want explicit control over the representative.
6. Make sure all nodes that will share one local dedup group are compatible.
7. Run `Export (slices + Prefab)`.
8. If the tool finds a reusable image in a common directory, decide whether to reuse the existing asset or replace it.

9. If an existing prefab already exists, review the incremental diff window and choose incremental apply or a fresh overwrite.

The Actual Export Order

This is the most important part of the tool to understand.

1. Collect exportable raster sources

The exporter first walks the PSD tree and collects raster sources.

- Regular image layers become raster candidates.
- Group layers become raster candidates only when `Merge for export` is enabled.
- Type layers are normally exported as UI text, not as PNG images.
- Layers with `Use external Prefab` do not rasterize.
- Layers with `Use custom image` do not rasterize.
- Layers with `Export = Off` are skipped entirely.
- Clipping masks are merged into their base result before the dedup pass.

At this point the exporter has a set of in-memory rasters, but it has not sliced them and has not written any PNG files yet.

2. Run local dedup inside the current export

The exporter then performs same-export dedup on the pending raster set.

- Only nodes with `Same-export dedup = On` can join a local dedup group.
- Nodes with `Same-export dedup = Off` are forced to export as unique files.
- Dedup is fingerprint-based and MAE-based.
- Transparent borders are trimmed first.
- The image is downscaled to the configured fingerprint size.
- The tool compares premultiplied RGBA fingerprints using mean absolute error.
- If MAE is at or below the configured threshold, the images are treated as identical for local dedup.

This means local dedup happens before any slicing or common-directory reuse.

3. Choose the representative node for each local dedup group

Every local dedup group is resolved to one virtual export job.

Representative selection rules:

- If one or more nodes in the group have `Primary dedup node = On`, the exporter picks the largest raster among those marked primary nodes.
- If no node is marked primary, the exporter picks the largest raster in the whole group as the image source.

Parameter resolution rules:

- If a primary node is chosen, both pixels and export parameters come from that chosen primary node.
- If no primary node is chosen, pixels come from the largest raster, but the export parameters may be synthesized from the whole group.

The synthesized rule matters:

- If any member in the group has `Slice / nine-slice = Off`, the whole virtual export job becomes non-sliced.
- If all members have slicing enabled but their effective nine-slice parameters differ, export aborts for safety.
- Common-directory dedup remains enabled for the group only when every member has `Common-directory dedup = On`.

In practice, if you want a stable outcome, either:

- align the slice settings across the whole duplicate group, or
- turn off same-export dedup for the outlier nodes, or
- explicitly mark one primary node and make the group behave consistently around that node

4. Slice or nine-slice the representative image

Only after the representative node is chosen does the exporter run slice detection.

- If `Slice / nine-slice = On`, the tool tries to detect slice borders.
- If detection succeeds, the exported PNG is the compressed nine-slice image.
- The detected border is written back to the Sprite importer as `spriteBorder`.
- If detection does not produce a valid border, the exporter falls back to a normal unsliced image.

Parameter source:

- Global values come from `Tools/PSD/Nine-slice settings...`
- Per-layer values override the global values when `Custom nine-slice params = On`

5. Run common-directory dedup

After local dedup and after slicing, the tool checks common directories.

- The tool loads every PNG under the folders listed in `PSD_CommonDirectories.json`.
- It fingerprints those PNGs with the same algorithm used for local dedup.
- It compares the final representative export image against the common-directory cache.
- If a match is found, the exporter reuses the existing common-directory PNG instead of writing a new one.

If `Detect larger image in common dirs` is enabled and the current export has a higher resolution than the matching shared image, the tool opens a choice dialog:

- `Use common directory image` : keep the existing shared PNG and reuse its path
- `Use current image (replace)` : overwrite the shared PNG on disk with the current higher-resolution image

If the option is disabled, the exporter simply reuses the existing shared image path without asking.

6. Write PNG files

If common-directory dedup does not reuse an existing shared image, the tool writes the final PNG file.

- Every local dedup group writes one PNG.
- All members of the group point to that same file path.
- Sprite import setup happens after writing.
- If auto naming is disabled and multiple results would produce the same basename, the exporter auto-renames the later ones with suffixes such as `_1` , `_2` , and so on.

7. Build sprites and prefab data

After images are finalized, the exporter:

1. imports the textures as `Sprite`
2. applies sprite borders for successful nine-slice results
3. builds the main prefab
4. optionally builds standalone prefabs for nodes marked `Export Prefab`

8. Run incremental prefab update when possible

If both of these already exist:

- the current main prefab
- the matching prefab map JSON

the tool does not immediately overwrite the prefab.

Instead it:

1. generates a temporary prefab from the current PSD state
2. compares the existing prefab against that temporary prefab
3. opens the export diff window
4. lets you apply an incremental patch or discard the old prefab and rebuild it

If there is no existing prefab, no prefab map, or no differences, the tool skips this review flow.

Incremental Update Behavior

The incremental diff window compares the existing prefab as dataset A and the newly generated temporary prefab as dataset B.

Difference classes:

- `Only in A` : a node exists only in the current prefab. This usually means it was added manually in Unity.
- `Only in B` : a node exists only in the PSD-generated result. This is treated as a new PSD node.
- `ParentChanged` : the same PSD node exists on both sides, but under a different parent.
- `ComponentChanged` : the same PSD node exists on both sides, but its serialized components differ. Name changes also fall into this class when `Compare name differences` is enabled.

Bottom-bar actions in the diff window:

- `Apply Incremental Selection` : applies only the selected structural and component changes.
- `Fresh Overwrite (Discard Existing Prefab)` : deletes the current prefab asset and regenerates it from scratch.
- `Cancel` : keeps the existing prefab unchanged.

Selection behavior:

- `Only in A` nodes can be kept or deleted.
- `ParentChanged` nodes can keep the Unity hierarchy or apply the PSD hierarchy.
- `ComponentChanged` nodes can keep the Unity-side components or apply the PSD-generated components.
- `Only in B` nodes are auto-added during incremental apply.

What `Compare name differences` changes:

- If enabled, object name differences are treated as real diffs.
- If disabled, name-only changes do not trigger component diff behavior.
- During incremental apply, names are only renamed when this option is enabled.

Setting Up Common Directories

Common-directory dedup only works after you register shared folders.

Use one of these approaches:

1. In the Unity Project window, right-click a folder and use `Assets/PsdToUnityUI/Add to Common Dir`.
2. Edit `Assets/PsdToUnityUI/EditorConfig/PSD_CommonDirectories.json` manually.

Notes:

- The tool scans registered folders recursively for `*.png`.
- The success popup recommends reopening the PSD Editor after changing the list.
- Common directories are for shared texture reuse, not for the local dedup pass.

Global Menus

Tools/PSD/Dedup settings...

This menu controls the fingerprint dedup algorithm used by both local dedup and common-directory dedup.

Option	What it does
<code>MAE threshold</code>	The maximum mean absolute error allowed before two images stop counting as the same. Lower is stricter. Higher is looser.
<code>Fingerprint size</code>	The downscale size used before comparison. Larger values are slower but more discriminating. Smaller values are faster but less precise.

Tools/PSD/Nine-slice settings...

These are the global defaults used whenever a layer does not enable custom nine-slice parameters.

Option	What it does
<code>Border inset (BORDER_INSET)</code>	Ignores a small edge margin before searching for slice boundaries, which helps avoid cuts exactly on color transitions.
<code>Adjacent pixel difference threshold (PIXEL_THRESHOLD)</code>	Maximum allowed RGBA difference between adjacent rows or columns for them to remain sliceable.
<code>Minimum contiguous cut zone (MIN_SAME_ZONE)</code>	Minimum continuous sliceable run required before an axis counts as strong enough for cutting.
<code>Max center columns (MIN_CENTER_COLS)</code>	Caps how many center columns remain after nine-slice compression.
<code>Max center rows (MIN_CENTER_ROWS)</code>	Caps how many center rows remain after nine-slice compression.

Home Screen Reference

These controls appear before any PSD is loaded.

Control	What it does
<code>Open PSD...</code>	Opens a PSD file into the editor.
<code>Recent Files</code>	Reopens recently used PSD files.
<code>Clear</code>	Clears the recent-file list only.
<code>Live composite preview (rebuild when layer visibility changes)</code>	When enabled, the canvas preview is rebuilt from current layer visibility. This is more accurate but slower. When disabled, the editor shows the PSD baked composite, which is faster but does not reflect visibility toggles.
<code>Default to TextMeshPro (type layers with no config or no useTextMeshPro field)</code>	Fallback used when a PSD or layer has no saved text export setting yet.
<code>Default to Slice / nine-slice (layers with no config or no sliceImage field)</code>	Fallback used when a PSD or layer has no saved slice setting yet.
<code>Use PSD node order in tree</code>	Changes only the display order of the left tree. It does not change the actual PSD data.

Left Tree and Canvas Reference

These are interaction controls rather than export options, but they are part of normal usage.

Control	What it does
Visibility dot	Toggles the layer visible state and refreshes the preview.
Click a row	Selects a layer.
<code>F2</code> or long-press on a name	Starts inline rename in the left tree.
<code>Enter</code> while renaming	Commits the rename.
<code>Esc</code> while renaming	Cancels the rename.
Up and Down arrows	Navigate visible tree rows.
Click in canvas	Selects the top matching layer under the cursor. Repeated clicks at the same spot cycle through overlapping hits.

Right Panel Reference

Basic controls

Control	What it does
Name	Renames the layer for exported hierarchy and saved config. Use <code>Restore PSD Names</code> if you want to reset back to the source PSD names.
Export	When off, the layer is skipped for both texture export and prefab node creation.
Visible	Updates the layer visible state in preview and persists that state into export config.

External prefab replacement

Control	What it does
Use external Prefab	Replaces this node with an existing prefab during export. No slicing, dedup, or normal image export happens for this node.
Prefab reference	The prefab asset to instantiate in place of the PSD node.
Reuse this PSD node position	Keeps the PSD node transform position when placing the external prefab.
Reuse this node size	Reapplies the PSD node size to the placed external prefab.

Image export controls

Control	What it does
Merge for export	Exports this group and all of its children as one image. The children are no longer exported separately.
Single export (no dedup)	Exports only this image node or merged group directly to disk. It skips both local dedup and common-directory dedup. It still applies nine-slice when enabled.
Export Prefab	Generates an additional standalone prefab for this node during export.
Export subtree now (slices + Prefab)	Exports only this subtree, including textures and standalone prefabs inside that subtree. It does not build the full PSD root prefab.
Use custom image	Uses a manually assigned sprite instead of rasterizing the PSD node. This skips slicing, dedup, and prefab replacement logic for the node.

Control	What it does
Custom Sprite	The sprite used when Use custom image is enabled.

UI type controls

Control	What it does
Type	Chooses which UGUI component should be attached on export. Supported values are <code>None</code> , <code>Button</code> , <code>ScrollBar</code> , <code>Slider</code> , <code>ScrollRect</code> , <code>Toggle</code> , <code>InputField(Legacy)</code> , <code>InputField(TMP)</code> , <code>Dropdown(Legacy)</code> , and <code>Dropdown(TMP)</code> .
Use TextMeshPro	For plain type layers, exports text as <code>TextMeshProUGUI</code> when enabled and legacy <code>Text</code> when disabled. For input fields and dropdowns, choose the specific Legacy or TMP type from the <code>Type</code> dropdown instead.

Dedup and slice controls

Control	What it does
Primary dedup node	Marks the node as a preferred source inside a local dedup group. If multiple nodes are marked, the largest marked raster wins.
Slice / nine-slice	Enables or disables the nine-slice detection path for this node. When off, the exporter writes a normal image.
Custom nine-slice params	Overrides the global <code>Tools/PSD/Nine-slice settings...</code> values for this layer only.
Border inset (BORDER_INSET)	Layer-local version of the global border inset.
Adjacent pixel diff threshold (0-255)	Layer-local version of the global adjacent pixel threshold.
Min same-zone run (MIN_SAME_ZONE)	Layer-local version of the global minimum contiguous cut zone.
Max center column shrink	Layer-local cap for the center width kept after nine-slice compression.
Max center row shrink	Layer-local cap for the center height kept after nine-slice compression.
Same-export dedup	Lets this node participate in the local dedup pass inside the current export job.
Common-directory dedup	Lets this node participate in the shared common-directory reuse pass.

Type-Specific Option Reference

None

No extra fields. The exporter does not attach a special UI component.

Button

No extra fields. The exporter attaches a `Button` component without additional child-name mapping.

ScrollBar

Control	What it does
ScrollBar direction	Sets Unity <code>Scrollbar.Direction</code> .
Handle child name	Resolves the child that should become <code>Scrollbar.handleRect</code> .

Slider

Control	What it does
Slider direction	Sets Unity <code>Slider.Direction</code> .
Fill Rect child name	Resolves the child that should become <code>Slider.fillRect</code> .
Handle Rect child name	Resolves the child that should become <code>Slider.handleRect</code> .

ScrollRect

Control	What it does
ScrollBg child name	Finds the child whose layout should be copied into a generated <code>Viewport</code> . The viewport is inserted right after that node.
Content child name	Finds the existing content node, reparents it under the generated viewport, and assigns it to <code>ScrollRect.content</code> .
Horizontal	Enables horizontal scrolling.
Vertical	Enables vertical scrolling.
HorizontalScrollbar child name	Finds a child <code>Scrollbar</code> and assigns it to <code>ScrollRect.horizontalScrollbar</code> .
VerticalScrollbar child name	Finds a child <code>Scrollbar</code> and assigns it to <code>ScrollRect.verticalScrollbar</code> .

Toggle

Control	What it does
Graphic child name	Finds the child graphic that should be assigned to <code>Toggle.graphic</code> .

InputField(Legacy)

Control	What it does
Text Component	Finds the <code>UnityEngine.UI.Text</code> child used as <code>InputField.textComponent</code> .
Placeholder	Finds the <code>UnityEngine.UI.Text</code> child used as <code>InputField.placeholder</code> .

InputField(TMP)

Control	What it does
Text Component	Finds the <code>TextMeshProUGUI</code> child used as <code>TMP_InputField.textComponent</code> .
Placeholder	Finds the <code>TextMeshProUGUI</code> child used as <code>TMP_InputField.placeholder</code> .
Text Viewport child name	Finds the viewport rect, assigns it to <code>TMP_InputField.textViewport</code> , adds <code>RectMask2D</code> if needed, and reparents the text and placeholder under it if necessary.

Dropdown(Legacy)

Control	What it does
Template child name	Finds the <code>RectTransform</code> assigned to <code>Dropdown.template</code> .
Caption Text child name	Finds the <code>UnityEngine.UI.Text</code> child assigned to <code>Dropdown.captionText</code> .
Item Text child name	Finds the <code>UnityEngine.UI.Text</code> child assigned to <code>Dropdown.itemText</code> . If needed, the exporter reparents it under the dropdown template content area.

Dropdown(TMP)

Control	What it does
Template child name	Finds the <code>RectTransform</code> assigned to <code>TMP_Dropdown.template</code> .
Caption Text child name	Finds the <code>TextMeshProUGUI</code> child assigned to <code>TMP_Dropdown.captionText</code> .
Item Text child name	Finds the <code>TextMeshProUGUI</code> child assigned to <code>TMP_Dropdown.itemText</code> . If needed, the exporter reparents it under the dropdown template content area.

Bottom Panel Reference

Control	What it does
Set all labels to TMP or Set all labels to Legacy	Batch-switches all type layers between TextMeshPro and legacy text.
Set all images to Slice / nine-slice or Set all images to raw export	Batch-switches all slice-eligible layers between sliced export and normal raw export.
Auto image naming	When enabled, filenames become <code>NodeName_LayerId.png</code> . When disabled, the base name is only <code>NodeName.png</code> , and collisions may be auto-renamed with numeric suffixes.
Compare name differences	Includes object name changes in the prefab diff and incremental patch logic.
Reload PSD	Reloads the current PSD from disk.
Restore PSD Names	Restores every renamed node back to the original names that came from the PSD when it was first opened.
Export (slices + Prefab)	Runs the full pipeline: local dedup, representative selection, slicing, common-directory dedup, sprite import, prefab generation, and incremental diff if applicable.
Export textures	Runs the image side of the pipeline only. No prefab is generated.
Image export folder	The Assets-root folder used for images. Actual PNGs go to <code><ImageExportFolder>/<PSDName>/</code> .
Prefab export folder	The Assets-root folder used for the main prefab and standalone node prefabs.
Navigate Project to output after export	Selects the output folder or prefab in the Unity Project window after export.
Detect larger image in common dirs	When common-directory dedup hits a matching lower-resolution shared PNG, opens a dialog so you can choose whether to reuse the shared PNG or replace it with the current higher-resolution export.
Clear image folder before export	Deletes the PSD-specific image output folder before the full export buttons run, which removes stale slices from older exports.

Practical Rules for Dedup Groups

If you want the pipeline to behave predictably, treat each local dedup group as one export contract.

Good practice:

- Keep `Same-export dedup` on only for truly identical images.
- If you need explicit control, mark one intended representative with `Primary dedup node`.
- Keep `Slice / nine-slice` aligned across the whole group.
- Keep custom nine-slice parameter values aligned across the whole group.

- **Keep** `Common-directory dedup` **aligned** across the whole group.

Common failure cases:

- Same fingerprint group, but different effective nine-slice parameters: export aborts.
- Same fingerprint group, but one node disables slice: the whole group exports unsliced.
- Same fingerprint group, but one node disables common-directory dedup: the whole group stops participating in common-directory dedup.
- `Use external Prefab` **or** `Use custom image` **on a node**: that node no longer contributes a normal raster export.

Short Version

If you only remember one thing, remember this:

The exporter does not slice every PSD node first and dedup later. It first deduplicates the raw raster candidates, chooses one representative export job per group, then slices that representative, then tries common-directory reuse, and only after that does it write PNGs and update prefabs.